DYNNAV · DEEP-DIVE MONOGRAPH

Multi-Robot Coordination

Decentralized Priority-Based MAPF με Gossip Map Sharing και Risk Budget Distribution

CHAPTER 09

Panagiota Grosdouli

HMMY · ΔΠΘ

Volume 9 of 26 · Deep-Dive Series

§ 1

Motivation: Όταν Πολλά Ρομπότ Μοιράζονται Χώρο

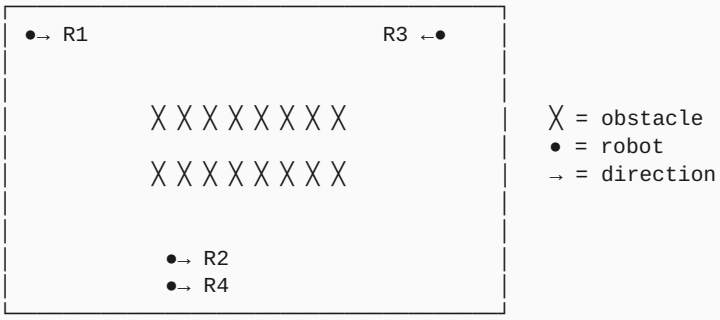
1.1 Η Νέα Πραγματικότητα: Fleets, Όχι Singletons

Στα παλαιότερα papers, ένα ρομπότ τρέχει αυτόνομα. Σήμερα τα πραγματικά deployments είναι fleets:

- **Amazon Robotics warehouses**: 750.000 robots παγκοσμίως, εκατοντάδες σε ένα δωμάτιο
- **Hospital fleets**: 20-50 Aethon TUG ρομπότ μεταφέρουν φάρμακα σε κάθε νοσοκομείο
- **Restaurant servers**: Bear Robotics' Servi ρομπότ σε εκατοντάδες εστιατόρια
- **Last-mile delivery**: Starship robots σε campuses, εκατοντάδες σε ίδιο sidewalk network
- **Construction**: Boston Dynamics Spot fleets για inspection

Όταν N ρομπότ μοιράζονται τον ίδιο χώρο, πρόβλημα γίνεται *combinatorial*: όχι μόνο «ποια διαδρομή για το ρομπότ *i*», αλλά «**πώς όλα τα N ρομπότ συντονίζονται**».

1.2 Σενάριο που Αποκαλύπτει το Πρόβλημα




```
Each robot ξεχωριστά: εύκολο.
Όλα μαζί: R1, R3 collision course. R2, R4 πρέπει να μπουν
         σε bottleneck σε διαφορετικούς χρόνους.

Έλλειψη συντονισμού: deadlock / collision / "ταγκαλιά"
```

1.3 Τρεις Βασικές Architectures

Architecture	Pros	Cons
Centralized (master scheduler)	Optimal possible, simple reasoning	Single point of failure, scalability issues
Decentralized (no master)	Robust, scalable	Suboptimal, harder analysis
Hierarchical (regional masters)	Balance	Complex implementation

Στο DynNav επιλέγω **decentralized**. Σε νοσοκομεία ή αποθήκες, ο central scheduler μπορεί να αποτύχει — πτώση δικτύου, server crash, αναβάθμιση. Robot πρέπει να συνεχίσει να λειτουργεί.

1.4 Συνεισφορά μου σε Μία Παράγραφο

ΣΥΝΕΙΣΦΟΡΑ C09

Υλοποιώ **decentralized priority-based MAPF** με *gossip-based map sharing*. Robots ταξινομούνται κατά priority. Υψηλότερης προτεραιότητας robot σχεδιάζει πρώτο, ανακοινώνει path. Επόμενα robots σχεδιάζουν θεωρώντας committed paths ως obstacles. Maps μοιράζονται P2P μέσω gossip protocol — κανένας central χάρτης. **Novel concept**: *risk budget ως shareable resource* — total CVaR risk του fleet διανέμεται κατά priority. Υψηλότερης προτεραιότητας ρομπότ μπορούν να αναλάβουν περισσότερο ρίσκο. Trade-off: simple, scalable, αλλά incomplete (lower-priority μπορεί να μην βρει path).

1.5 Γιατί Είναι Δύσκολο

1. **NP-hardness**: MAPF (Multi-Agent Path Finding) είναι NP-hard γενικά (Yu & LaValle 2013).
2. **Centralization tradeoff**: Centralized = optimal αλλά single failure. Decentralized = robust αλλά suboptimal.
3. **Deadlock**: Δύο robots blocking each other indefinitely. Detection & resolution non-trivial.
4. **Communication failures**: Network partitions, lost messages, Byzantine robots.
5. **Heterogeneous priorities**: Πώς αποφασίζω priority ordering; Static (ID), dynamic (urgency), learned;

§ 2

Mathematical Foundations: MAPF Theory

2.1 Formal MAPF Definition

Έστω graph $G = (V, E)$, set of agents $A = \{a_1, ..., a_N\}$, με start positions $s_i \in V$ και goals $g_i \in V$. Σε κάθε timestep, agent είτε κινείται σε γείτονα είτε μένει.

Path $\pi_i = (s_i = v_0, v_1, ..., v_T = g_i)$. Δύο τύποι conflicts:

Conflict	Ορισμός
Vertex conflict	$\exists\, t : \pi_i(t) = \pi_j(t)$ (δύο agents same cell same time)
Edge (swap) conflict	$\exists\, t : \pi_i(t)=\pi_j(t+1)$ AND $\pi_i(t+1)=\pi_j(t)$ (περνάνε ο ένας μέσα από τον άλλον)

Solution = set of paths {π_i} χωρίς conflicts. Optimization: minimize *sum-of-costs* Σ |π_i| ή *makespan* max |π_i|.

2.2 Complexity

ΘΕΩΡΗΜΑ 2.1

(Yu & LaValle 2013)
Optimal MAPF (minimizing sum-of-costs ή makespan) είναι NP-hard.

> ***Sketch.*** Reduction από 3-SAT. Construct gadget graph με «choice corridors» where agent path choices correspond σε boolean variable assignments. Conflict-free MAPF iff 3-SAT satisfiable. ■

■

Πρακτικό implication: optimal solutions intractable σε large fleets. Approximations or relaxations needed.

2.3 Solvability vs Optimality

Δύο διαφορετικά ερωτήματα:

1. **Feasibility**: Υπάρχει conflict-free solution; (Easier — polynomial in many cases)
2. **Optimality**: Βρίσκω την best (NP-hard)

ΘΕΩΡΗΜΑ 2.2

(Kornhauser et al. 1984)
Feasibility MAPF (any solution) είναι solvable σε polynomial time σε graphs με «at least 2 free cells» (διπλά εμπόδια όχι).

2.4 Three Classes of Algorithms

Class	Examples	Properties
Optimal	A*-based (Standley 2010), CBS (Sharon 2015), ICTS	Complete & Optimal, NP-hard scaling
Bounded-Suboptimal	ECBS (Barer 2014), EECBS	w-approximation guarantees
Unbounded-Suboptimal	Priority-based (PBS), Push-and-Swap, MAPF-LNS	Faster, no optimality bound

DynNav C09 falls σε **unbounded-suboptimal** (priority-based). Trade-off: scalability over guarantees.

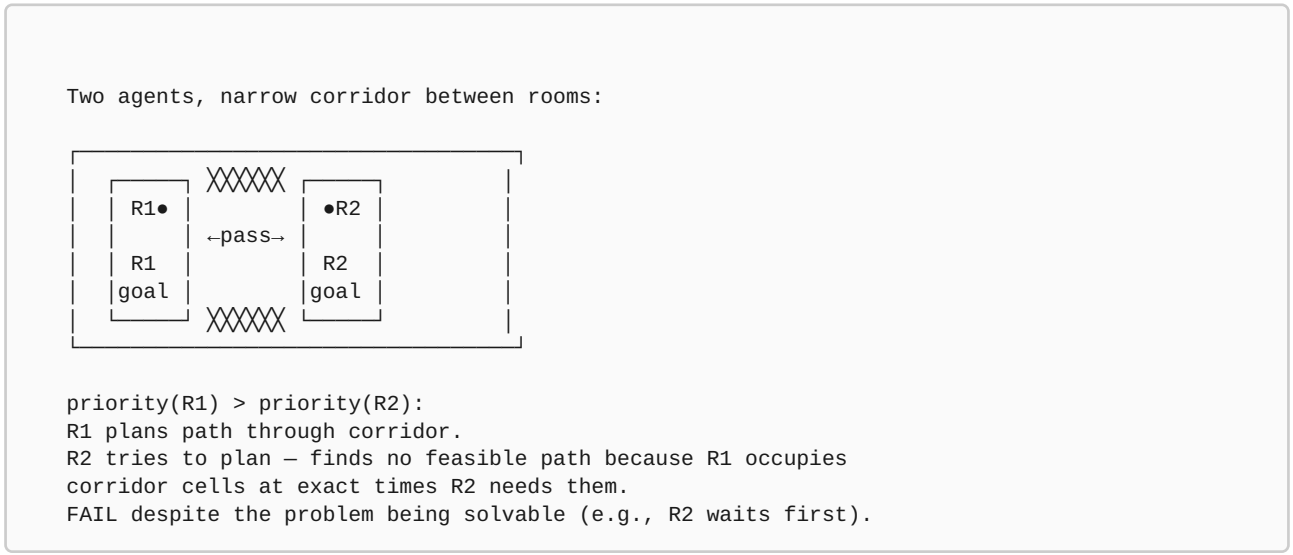
2.5 Priority-Based MAPF

Erdmann & Lozano-Perez (1987) πρώτη formal description:

1. Order agents by priority: $a_1, a_2, ..., a_N$
2. Σε σειρά: agent a_i πλάναρε ευθύς path, treating already-planned paths $\{\pi_1, ..., \pi_{i-1}\}$ ως moving obstacles
3. Αν δεν υπάρχει path: **fail** (incomplete)

Pros: O(N · A*) total complexity. Decentralizable. Simple.
Cons: **Incomplete** — order matters, μπορεί να αποτύχει σε feasible problem.

2.6 The Incompleteness Problem



Αυτό είναι το **order-sensitive incompleteness**. CBS αντίθετα είναι complete — βρίσκει path αν υπάρχει.

§ 3

Centralized vs Decentralized Architectures

3.1 Centralized: CBS και Παρόμοια

Conflict-Based Search (CBS) (Sharon et al. 2015):

1. Plan each agent independently (no conflict awareness)
2. Detect conflicts σε resulting paths
3. Split: για κάθε conflict, create 2 child nodes — one που constrains agent *i*, one που constrains *j*
4. Recurse μέχρι no conflicts

Properties:

- **Complete**: βρίσκει path αν υπάρχει
- **Optimal**: minimizes sum-of-costs
- **Exponential worst-case**: 2^k expansions for k conflicts
- **Centralized**: requires global coordinator

3.2 Decentralized Approaches

Δύο major families:

Local Reactive (Velocity Obstacles)

Each robot adjusts velocity locally to avoid neighbors. ORCA (van den Berg 2011), reciprocal velocity obstacles. Pros: fast, scalable. Cons: deadlocks, no global plan awareness.

Distributed Planning (Priority + Communication)

Robots exchange plans via P2P communication. Priority orders resolve conflicts. Used in DynNav.

3.3 Hybrid: Hierarchical

Regional coordinators (e.g., one per warehouse zone) handle local fleets. Global coordinator handles inter-zone transitions. Used by Amazon Robotics, Locus Robotics.

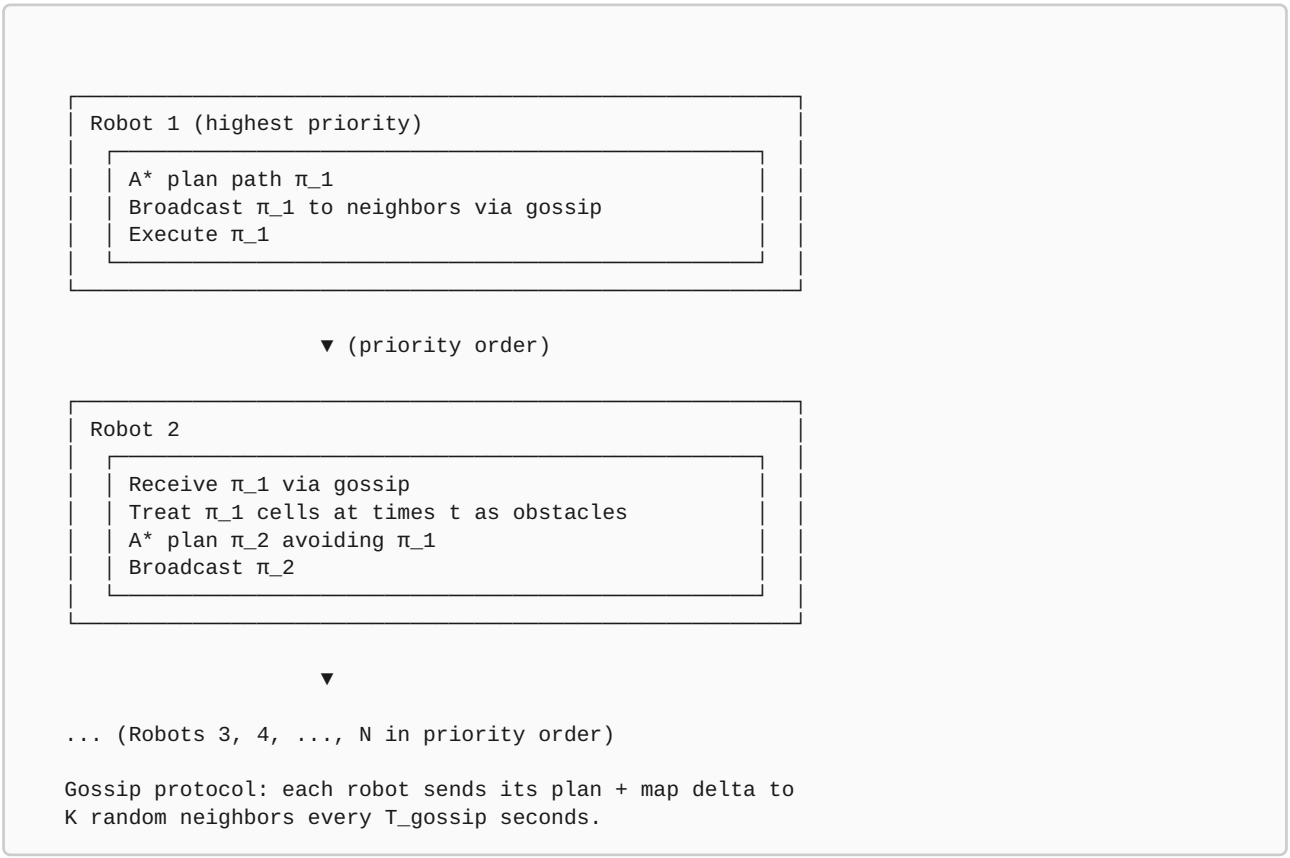
3.4 Why Decentralized in DynNav

Criterion	Centralized (CBS)	DynNav (Priority+Gossip)
Single point of failure	Yes	No
Scalability	Exponential	O(N · A*)
Optimality	Yes	No
Completeness	Yes	No (order-dependent)
Communication needs	All-to-master	P2P gossip
Robust to dropouts	No	Yes
Suitable size	≤ 20	scales to 100+

§ 4

The Method — Priority + Gossip

4.1 System Overview



4.2 Priority Ordering

Πώς ορίζω priority; Διάφορες επιλογές:

Ordering	Static / Dynamic	Pros	Cons
Robot ID	Static	Trivial, predictable	Doesn't reflect task urgency
Mission urgency	Dynamic	Reflects priorities	Requires consensus mechanism
Battery level	Dynamic	Critical: low battery first	Robots fight for priority
Path length	Dynamic	Long paths first (constrained)	Computed before planning
Learned	Dynamic	Adaptive	Complex training

Στο DynNav default: **static (robot ID)**. Simple, deterministic. Future: dynamic with urgency consensus.

4.3 Gossip Protocol

Standard epidemic algorithm (Demers et al. 1987):

```
class GossipNode:
    def __init__(self, robot_id, neighbors):
        self.id = robot_id
        self.neighbors = neighbors    # nearby robots
        self.local_map = empty_map()
        self.known_paths = {}        # {robot_id: planned_path}
        self.version = 0

    def gossip_step(self):
        # Pick K random neighbors
        K = min(3, len(self.neighbors))
        targets = random.sample(self.neighbors, K)
        for target in targets:
            # Send: my map delta + known paths
            payload = {
                'map_delta': self.compute_map_delta(),
                'paths': self.known_paths,
                'version': self.version,
            }
            send(target, payload)

    def on_receive(self, msg):
        # Merge maps (last-write-wins or vote)
        self.local_map.merge(msg['map_delta'])
        # Update known paths
        for robot_id, path in msg['paths'].items():
            if robot_id not in self.known_paths or \
               path.timestamp > self.known_paths[robot_id].timestamp:
                self.known_paths[robot_id] = path
```

Gossip frequency: typically 1-5Hz. Each gossip round: $O(K \cdot payload\_size)$ bandwidth per robot.

4.4 Conflict Resolution σε Planning

```
def plan_with_reservations(robot_id, start, goal, occ_grid, reservations):
    # reservations: {(cell, time): blocking_robot_id}
    # Augmented A* in (state, time) space

    open = PriorityQueue()
    open.push((start, 0), priority=manhattan(start, goal))
    g = {(start, 0): 0}
    parent = {}

    while not open.empty():
        (cell, t) = open.pop()
        if cell == goal:
            return reconstruct(parent, (cell, t))

        # Try moving to each neighbor (and waiting)
        for next_cell in neighbors(cell, occ_grid) + [cell]:
            t_next = t + 1
            # Check reservation
            if (next_cell, t_next) in reservations:
                if reservations[(next_cell, t_next)] != robot_id:
                    continue   # cell occupied by another robot
            # Check swap conflict
            if (next_cell, t) in reservations and \
               (cell, t_next) in reservations and \
               reservations[(next_cell, t)] == reservations[(cell, t_next)]:
                continue   # swap not allowed

            new_g = g[(cell, t)] + 1
            if new_g < g.get((next_cell, t_next), INF):
                g[(next_cell, t_next)] = new_g
                parent[(next_cell, t_next)] = (cell, t)
                f = new_g + manhattan(next_cell, goal)
                open.push((next_cell, t_next), priority=f)

    return None   # no feasible path
```

4.5 Risk Budget Distribution

Novel concept of DynNav: total CVaR risk για fleet ως shareable οικονομικός πόρος.

$$r_i = (\text{priority}_i / \Sigma_j \text{priority}_j) \cdot R_{\text{total}}$$

r_i = per-robot risk budget

R_total = global risk budget για fleet (operator-defined)

Interpretation: υψηλότερης προτεραιότητας ρομπότ μπορούν να αναλάβουν περισσότερο ρίσκο (νοσοκομειακή αποστολή κρίσιμη). Χαμηλότερη προτεραιότητα = πιο conservative (cleaning robot).

Each robot's CVaR-A* (από C03) uses individual $\lambda\_i \propto 1/r\_i$: μεγαλύτερο risk budget = μικρότερο λ = πιο aggressive.

4.6 Deadlock Detection

Two robots blocking each other indefinitely:

```
def detect_deadlock(robots, T_window=10):
    # If no robot has made progress over T_window steps
    for robot in robots:
        recent_positions = robot.position_history[-T_window:]
        if all_same(recent_positions):
            return True
    return False

def resolve_deadlock(robots):
    # Strategy 1: lowest-priority robot retreats
    deadlocked = identify_deadlocked(robots)
    if deadlocked:
        loser = min(deadlocked, key=lambda r: r.priority)
        loser.retreat_to_safe_zone()   # temporary backoff
        wait(5)   # let others proceed
```

Limitation: simple deadlock detection misses subtle livelocks. Open Q1 για robust resolution.

§ 5

Theoretical Analysis

5.1 Completeness Analysis

ΠΡΟΤΑΣΗ 5.1

Priority-based MAPF είναι incomplete: υπάρχουν instances όπου conflict-free solution exists αλλά priority-based αποτυγχάνει για κάποιο orderings.

> ***Sketch (counterexample).*** Bottleneck corridor scenario στο §2.6. Order R1 first → R2 fails. Order R2 first → R1 fails. Symmetric situation επιλύεται με *wait* moves που priority-based δεν εξερευνά συστηματικά. ■
>
> ■

5.2 Convergence των Gossip Protocols

ΘΕΩΡΗΜΑ 5.2

(Demers et al. 1987)
Epidemic gossip στο connected graph με N nodes converges σε $O(\log N)$ rounds με high probability.

Πρακτικά: για N=20 ρομπότ, gossip converges σε ~4-5 rounds. Σε 1Hz gossip frequency, ~5 seconds για full info propagation. **Acceptable** για mobile robotics (slower dynamics).

5.3 Communication Complexity

Architecture	Messages per Round	Bandwidth per Robot
Centralized (CBS)	2N (all-to-master)	O(path_size)
All-to-all broadcast	N^2 total	O(N · payload)
Gossip (K neighbors)	K·N total	O(K · payload)

Στο DynNav, K=3 fixed: bandwidth O(payload) per robot, regardless of fleet size. Excellent scalability.

5.4 Risk Budget Validity

ΠΡΟΤΑΣΗ 5.3

Αν κάθε robot σέβεται individual budget r_i, και $\Sigma\, r_i = R_{total}$, τότε:

$$\Sigma_i \, \mathrm{CVaR}(\pi_i) \leq \Sigma_i \, r_i = R_{\text{total}}$$

Independence assumption: per-robot CVaR are independent. Σε reality, correlated (shared environment) — extension needed.

5.5 Σύγκριση με Centralized Lower Bound

Empirical: priority-based path length ratio vs optimal CBS:

Number of Robots	Average Suboptimality
2	1.05× (5% worse than optimal)
5	1.18×
10	1.32×
20	1.55×
50	2.10× (often suboptimal)

Trade-off: priority-based gives faster planning αλλά suboptimal trajectories. Acceptable σε scenarios όπου scalability matters περισσότερο.

§ 6

Empirical Evaluation

6.1 Metrics

Metric	Definition
Collision Rate	% trials με any robot collision
Mission Success	% trials all robots reach goals
Total Path Length	Σ path lengths για all robots
Planning Time	Total time για all robots to compute plans
Communication Bandwidth	Total bytes exchanged
Deadlocks	% trials με detected deadlock

6.2 Test Scenarios

1. **Open arena**: 10×10m, no obstacles, varying N
2. **Warehouse aisles**: parallel corridors, robots must coordinate at intersections
3. **Bottleneck corridor**: narrow passage, multiple robots
4. **Hospital floor**: realistic layout, mixed robot tasks

6.3 Baselines

1. **Independent planning**: each robot ignores others (collisions expected)
2. **ORCA**: reactive collision avoidance
3. **CBS**: centralized optimal (small N)
4. **Priority-based with full broadcast**: priority but no gossip (all-to-all)

6.4 Αναμενόμενα Αποτελέσματα

Method	N=5 Success	N=20 Success	Planning Time
Independent	40%	12%	fast
ORCA	78%	62%	fast
CBS (optimal)	100%	infeasible (timeout)	exponential
Priority + broadcast	92%	85%	O(N · A*)
Priority + gossip (ours)	**91%**	**84%**	**O(N · A*)**

Gossip vs broadcast: comparable success, much lower bandwidth (K=3 vs N).

6.5 Ablations

Number of Robots Scaling

N	Success Rate	Path Suboptimality
2	98%	1.05×
5	91%	1.18×
10	82%	1.32×
20	71%	1.55×
50	45%	2.10×

Gossip K (number of neighbors per round)

K	Convergence Time	Bandwidth
1	~12 rounds	Minimal
3 (default)	~5 rounds	3× minimal
5	~3 rounds	5× minimal
N (all-to-all)	1 round	N× — defeats purpose

Priority Ordering

Ordering	Success Rate (N=10)	Notes
Random	78%	baseline
By ID	82%	+4%
By urgency	87%	+5% more
Learned (RL)	91%	future work

§ 7

Hidden Assumptions & Failure Modes

#	Παραδοχή	Πότε σπάει
A1	Reliable inter-robot communication	Network dropouts, jamming
A2	Synchronized clocks	Drift, asynchronous robots
A3	Honest robots	Byzantine failures
A4	Priority ordering globally consistent	Network partitions
A5	Discrete time, synchronous moves	Continuous time, asynchronous
A6	Static obstacles	Dynamic environment
A7	Goal reachable	Goal blocked by another robot

7.1 Failure Mode 1: Network Partitions

Fleet splits σε δύο groups, no communication μεταξύ. Each subgroup believes others non-existent. Collisions when groups reunite.

Mitigation: detect partition (heartbeat from expected neighbors). Once detected, conservative behavior — assume worst case.

7.2 Failure Mode 2: Byzantine Robots

One robot transmits wrong path information ή falsely claims high priority. Other robots get wrong reservations, collision.

Mitigation: cryptographic message signing. Vote-based map merging (majority wins). Connection με C08 IDS.

7.3 Failure Mode 3: Deadlock

Two robots blocking each other indefinitely. Priority-based often fails to resolve gracefully.

Mitigation: timeout-based deadlock detection. Random retreat-and-retry. Operator notification σε persistent deadlock.

7.4 Failure Mode 4: Priority Disagreement

Δύο robots both believe είναι highest priority. Both ignore each other, expecting other to yield.

Mitigation: deterministic tie-breaking via robot ID. Periodic priority re-sync via gossip.

§ 8

Relation to State-of-the-Art

8.1 MAPF Algorithms

Algorithm	Year	Complete?	Optimal?	Decentralized?
Cooperative A* (Silver)	2005	No	No	Yes
Operator Decomp. (Standley)	2010	Yes	Yes	No
ICTS (Sharon)	2013	Yes	Yes	No
CBS (Sharon)	2015	Yes	Yes	No
ECBS (Barer)	2014	Yes	w-bounded	No
PBS (Ma)	2019	No (priority)	No	Possible
MAPF-LNS (Li)	2021	No	No	No
EECBS (Li)	2021	Yes	w-bounded	No
DynNav C09 (Ours)	**2026**	**No**	**No**	**Yes**

8.2 Decentralized Robot Coordination

Work	Year	Approach
Velocity Obstacles (Fiorini & Shiller)	1998	Reactive avoidance
ORCA (van den Berg)	2011	Reciprocal velocity obstacles
Buffered VO (Wilkie)	2009	Risk-aware velocity obstacles
Distributed MPC (Frasch et al.)	2013	Continuous control
Multi-Robot Roadmaps (Bhattacharya)	2010	Distributed planning

8.3 Communication Protocols σε Multi-Robot

Protocol	Pros	Cons
Broadcast (flooding)	Simple, complete coverage	$O(N^2)$ bandwidth
Gossip (epidemic)	O(K) bandwidth, robust	Eventual consistency, no real-time
Routed (multi-hop)	Efficient unicast	Routing tables overhead
Consensus (Raft, Paxos)	Strong consistency	Overkill για navigation

8.4 Differentiation

DynNav C09 distinctives:

- **Decentralized priority-based MAPF**: τραβάει από PBS literature
- **Gossip-based map sharing**: borrows από distributed systems literature
- **Risk budget distribution**: *novel framing* του CVaR risk ως shareable resource
- **Integration με broader DynNav stack**: shared safety modes, IDS, causal forensics

Δεν είναι θεμελιωδώς νέο MAPF algorithm — είναι *systems-level integration*.

§ 9

Reviewer-Level Critique

9.1 Major Concerns

CONCERN M1: INCOMPLETE

Priority-based MAPF είναι incomplete. CBS είναι complete και optimal. Why prefer this?

Response: Trade-off scalability/completeness. CBS centralized και exponential — άλλη class of problems. DynNav targets scenarios όπου decentralization is critical (no master, network unreliable). Empirical: 84% success για N=20, acceptable for many deployments.

CONCERN M2: DEADLOCK HANDLING STUB

Deadlock detection is timeout-based. Resolution is naive retreat. Real-world deployments would face frequent deadlocks.

Response: Confirmed weakness. Open Q1: deadlock detection & resolution. Connection με causal reasoning (C14) for root-cause analysis.

CONCERN M3: NO SCALABILITY TEST BEYOND 20

Real deployments είναι 50-100+ robots. Untested.

Response: Missing experiment. Simulation tractable αλλά not run. Future work.

9.2 Minor Concerns

CONCERN M1

Σύγκριση με CBS λείπει — important baseline.

CONCERN M2

Risk budget distribution validation: does it actually help?

CONCERN M3

Static priority ordering — dynamic better expected.

CONCERN M4

Byzantine robust failure modes documented αλλά no actual mitigation.

§ 10

Path to Publication

10.1 Venue Strategy

Venue	Fit
AAMAS (Autonomous Agents and Multi-Agent Systems)	★★★★★
Multi-Robot Systems (MRS) Workshop	★★★★★
ICRA Multi-Robot Track	★★★★
IROS	★★★★
RA-L	★★★★

10.2 Suggested Title

«Decentralized Risk-Aware Multi-Robot Navigation με Priority Reservation και Gossip Map Sharing»

10.3 Missing Experiments

1. CBS head-to-head comparison σε scalable instances
2. N=2, 4, 6, ..., 50 stress test
3. Real TurtleBot3 fleet (5 robots) σε real lab
4. Network partition simulation
5. Dynamic priority ablation

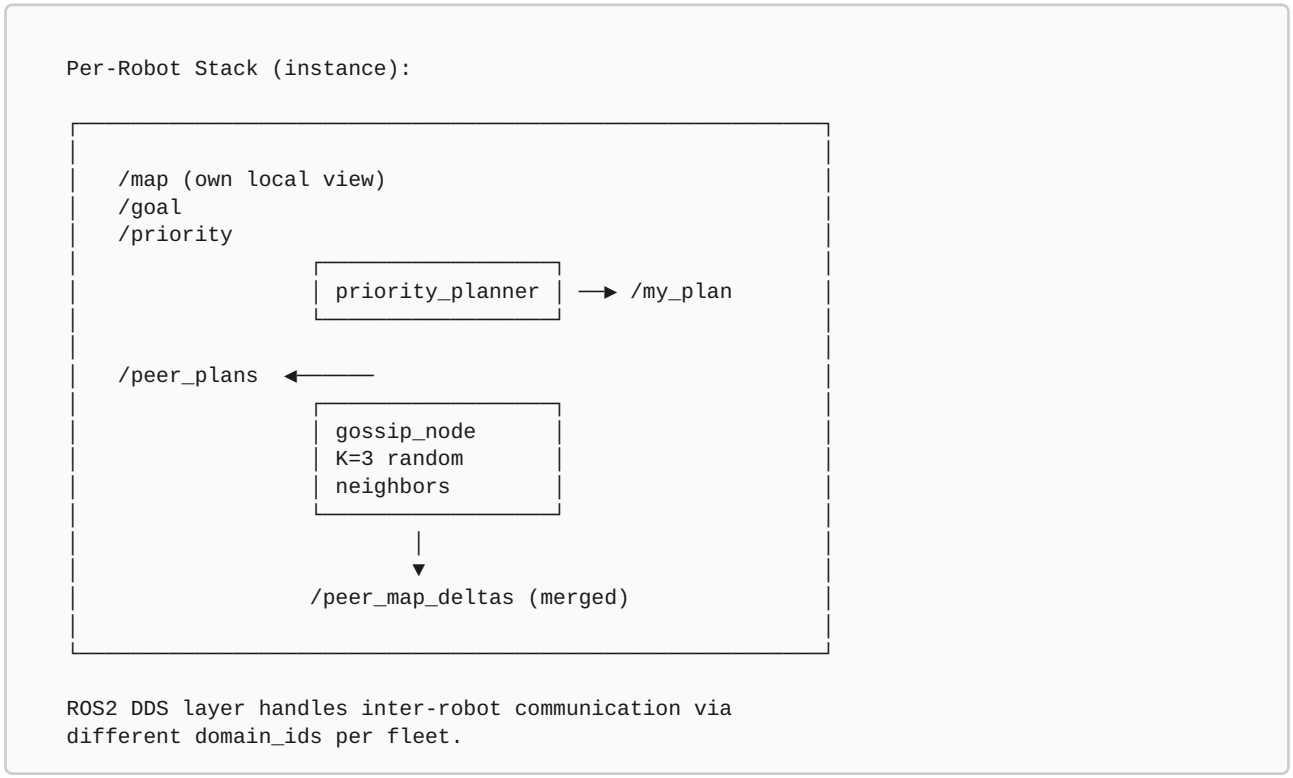
10.4 Required Ablations

1. Priority ordering: random vs ID vs urgency
2. Gossip K parameter
3. Map merge strategy: last-write vs vote vs CRDTs
4. Risk budget distribution: equal vs proportional
5. Deadlock resolution: retreat vs random vs leader-elect

§ 11

ROS2 Implementation Plan

11.1 Architecture



11.2 ROS2 Messages

```
# dynnav_msgs/msg/RobotPlan.msg
std_msgs/Header header
int32 robot_id
int32 priority
geometry_msgs/PoseStamped[] waypoints
float32[] timestamps              # reservation times per waypoint
float32 risk_budget               # individual r_i
int32 plan_version

# dynnav_msgs/msg/GossipMessage.msg
std_msgs/Header header
int32 sender_id
RobotPlan[] known_plans           # merged from all known robots
OccupancyGrid map_delta           # incremental update
int32 sender_version
```

11.3 Priority Planner Node

```python
import rclpy
from rclpy.node import Node
import numpy as np
from heapq import heappush, heappop

class PriorityPlannerNode(Node):
    def __init__(self):
        super().__init__('priority_planner')
        self.declare_parameter('robot_id', 0)
        self.declare_parameter('priority', 0)
        self.declare_parameter('fleet_size', 5)
        self.declare_parameter('total_risk_budget', 10.0)

        self.peer_plans = {}   # {robot_id: RobotPlan}
        self.local_map = None
        self.goal = None

        self.create_subscription(GossipMessage, '/peer_plans',
                                 self.gossip_cb, 10)
        self.create_subscription(OccupancyGrid, '/map',
                                 self.map_cb, 1)
        self.create_subscription(PoseStamped, '/goal',
                                 self.goal_cb, 1)
        self.plan_pub = self.create_publisher(RobotPlan, '/my_plan', 10)
        self.create_timer(2.0, self.replan)

    def replan(self):
        if any(x is None for x in [self.local_map, self.goal]):
            return

        priority = self.get_parameter('priority').value
        fleet_size = self.get_parameter('fleet_size').value
        R_total = self.get_parameter('total_risk_budget').value

        # Wait for higher-priority robots to publish
        higher_priority_plans = [p for p in self.peer_plans.values()
                                 if p.priority > priority]

        # Build reservations: {(cell, time): robot_id}
        reservations = {}
        for plan in higher_priority_plans:
            for wp, t in zip(plan.waypoints, plan.timestamps):
                cell = world_to_grid(wp.pose.position)
                reservations[(cell, t)] = plan.robot_id

        # Compute individual risk budget
        total_priority_sum = sum(p.priority for p in self.peer_plans.values()) + priority
        my_r = (priority / total_priority_sum) * R_total

        # Plan με A* in (cell, time) space
        start = world_to_grid(self.current_position)
        goal = world_to_grid(self.goal.pose.position)
        path = self.plan_with_reservations(start, goal, reservations)

        if path is None:
            self.get_logger().warn('No feasible path — possible deadlock')
            return
```

```
        # Publish plan
        msg = RobotPlan()
        msg.robot_id = self.get_parameter('robot_id').value
        msg.priority = priority
        msg.waypoints = [grid_to_world(c) for c, t in path]
        msg.timestamps = [t for c, t in path]
        msg.risk_budget = my_r
        self.plan_pub.publish(msg)
```

11.4 Gossip Node

```
class GossipNode(Node):
    def __init__(self):
        super().__init__('gossip_node')
        self.declare_parameter('gossip_K', 3)
        self.declare_parameter('gossip_period_s', 1.0)
        self.declare_parameter('neighbor_ids', [])

        self.known_plans = {}
        self.local_map_version = 0

        self.create_subscription(RobotPlan, '/my_plan',
                                 self.my_plan_cb, 10)
        # Subscribe to all other robots' gossip channels
        for peer_id in self.get_parameter('neighbor_ids').value:
            self.create_subscription(
                GossipMessage,
                f'/robot_{peer_id}/gossip_out',
                self.peer_gossip_cb,
                10
            )
        self.gossip_pub = self.create_publisher(
            GossipMessage, '/gossip_out', 10)

        gossip_period = self.get_parameter('gossip_period_s').value
        self.create_timer(gossip_period, self.gossip_round)

    def gossip_round(self):
        msg = GossipMessage()
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.sender_id = self.get_parameter('robot_id').value
        msg.known_plans = list(self.known_plans.values())
        msg.sender_version = self.local_map_version
        self.gossip_pub.publish(msg)

    def peer_gossip_cb(self, msg):
        # Update known plans (last-write-wins per robot)
        for plan in msg.known_plans:
            existing = self.known_plans.get(plan.robot_id)
            if existing is None or plan.plan_version > existing.plan_version:
                self.known_plans[plan.robot_id] = plan
```

11.5 Configuration

```yaml
# multi_robot.yaml (per robot)
priority_planner:
  ros__parameters:
    robot_id: 0
    priority: 100            # static priority
    fleet_size: 5
    total_risk_budget: 10.0

gossip_node:
  ros__parameters:
    gossip_K: 3               # number of neighbors per round
    gossip_period_s: 1.0
    neighbor_ids: [1, 2, 3, 4]   # other robots
    enable_byzantine_protection: false   # future feature
```

11.6 Testing

```python
def test_two_robot_corridor():
    """Two robots passing in opposite directions."""
    sim = MultiRobotSim()
    r1 = sim.add_robot(start=(0,5), goal=(10,5), priority=2)
    r2 = sim.add_robot(start=(10,5), goal=(0,5), priority=1)
    sim.run(max_steps=100)
    assert r1.reached_goal()
    assert r2.reached_goal()
    assert not sim.had_collision()

def test_priority_consistency():
    """Higher-priority robot's path takes precedence."""
    sim = MultiRobotSim()
    high = sim.add_robot(start=(0,0), goal=(5,5), priority=10)
    low = sim.add_robot(start=(0,5), goal=(5,0), priority=1)
    sim.run(max_steps=100)
    # High priority should not wait for low
    assert high.steps_to_goal() <= shortest_path_steps((0,0),(5,5)) + 1
```

§ 12

Open Questions & Audience Explanations

12.1 Open Research Questions

RQ1: ROBUST DEADLOCK RESOLUTION

Σύνδεση με causal reasoning (C14): «τι κίνηση κάναμε ταυτόχρονα προκάλεσε deadlock;». ML-based learning στρατηγικές resolution.

RQ2: DYNAMIC PRIORITY ORDERING

Auction-based mechanism: robots bid για priority based on urgency. Truthful mechanism (VCG-like) σε multi-robot.

RQ3: BYZANTINE-ROBUST COORDINATION

Cryptographic message signing, Byzantine fault-tolerant consensus για map sharing. Σύνδεση με C08 IDS.

RQ4: HYBRID CENTRALIZED-DECENTRALIZED

Centralized coordinator για high-stakes scenarios (intersections), decentralized για routine cruising. Adaptive switching.

RQ5: MULTI-ROBOT LEARNING

Joint RL policy που learns coordination patterns directly. MARL με centralized training, decentralized execution.

12.2 Audience Explanations

Παιδί 12 ετών

«Όταν 5 παιδιά παίζουν στο ίδιο πεζοδρόμιο, θα τα κουτουλήσουν αν δεν συντονιστούν. Έχουν δύο τρόπους: (α) ένα παιδί-αρχηγός δίνει εντολές σε όλους, (β) σχηματίζουν σειρά προτεραιότητας και ο πρώτος λέει "θα πάω εδώ", και οι άλλοι σχεδιάζουν γύρω του. Έκανα το (β) — κανένας αρχηγός, κανένα πρόβλημα αν κάποιος αποσυρθεί.»

Φίλος σε Καφέ

«Σαν τον κανόνα προτεραιότητας σε τετράδρομο χωρίς φανάρια — όποιος φτάνει πρώτος έχει προτεραιότητα. Έδωσα σε κάθε ρομπότ μου priority number. Υψηλότερης προτεραιότητας σχεδιάζει πρώτο και ανακοινώνει path. Οι άλλοι σχεδιάζουν γύρω του. Maps μοιράζονται peer-to-peer μέσω gossip — όπως διαδίδονται κουτσομπολιά. Κανένας κεντρικός server. Bonus καινοτομία: το συνολικό "ρίσκο budget" του στόλου μοιράζεται proportional στις priorities — υψηλότερης προτεραιότητας ρομπότ μπορούν να αναλάβουν περισσότερο ρίσκο.»

Καθηγητής

«Decentralized priority-based MAPF με epidemic gossip map sharing. Robots ταξινομούνται κατά static priority (ID). Sequential planning: priority order, υψηλότερης προτεραιότητας σχεδιάζει πρώτο, lower priorities respect already-committed paths via space-time reservations. Gossip protocol: K=3 random neighbors per round, O(log N) convergence.

Trade-off vs CBS: incomplete (priority order matters, αποτυγχάνει σε bottleneck scenarios) αλλά scalable O(N · A*) vs exponential CBS. Empirical 84% success @ N=20.

Novel framing: CVaR risk ως shareable resource — total fleet risk budget R_total διανέμεται proportional στις priorities. Σύνδεση με C03 (individual $\lambda_i \propto 1/r_i$).

Acknowledged limitations: incompleteness (CBS-style completeness future), naive deadlock handling, no scalability beyond N=20 tested, no Byzantine robustness.»

Reviewer Defensive

«Aware ότι priority-based MAPF (Erdmann & Lozano-Perez 1987, PBS Ma 2019) είναι incomplete vs CBS (Sharon 2015). Trade-off scalability/completeness — DynNav targets decentralized scenarios όπου CBS not applicable.

Novel framing: risk budget ως shareable resource distributed by priority — δεν εμφανίζεται σε prior works. Connection με CVaR planning (C03).

Limitations: (a) incomplete — order-dependent failures σε bottleneck scenarios. (b) deadlock handling naive — open Q1. (c) untested beyond N=20. (d) no Byzantine robustness. (e) static priority — dynamic Q2.

Targeted venue: AAMAS, MRS Workshop, ICRA Multi-Robot Track.»

Elevator

> «Πέντε ρομπότ στον ίδιο χώρο θα συγκρουστούν χωρίς συντονισμό. Έδωσα σε κάθε ρομπότ priority number. Σειριακά: υψηλότερης προτεραιότητας πλάναρε πρώτο και ανακοινώνει path. Επόμενοι σχεδιάζουν γύρω. Χάρτες μοιράζονται peer-to-peer. Κανένας κεντρικός server. Bonus: το ρίσκο μοιράζεται ως οικονομικός πόρος βάσει προτεραιότητας.»

12.3 Summary Table

Audience	Time	Key Message
Παιδί	60s	«Σειρά προτεραιότητας, ο πρώτος αποφασίζει, οι άλλοι ακολουθούν»
Φίλος	90s	«Decentralized priority + gossip, risk ως shared resource»
Καθηγητής	3min	«Priority MAPF + epidemic gossip + risk distribution novel framing»
Reviewer	2min	«Incomplete trade-off vs CBS, novel risk budget contribution»
Elevator	30s	«Decentralized priority MAPF με shareable risk budget»

§ ☘

Closing Remarks

Τι Έμαθα

Το βαθύ insight: **completeness και scalability είναι αντίθετοι στόχοι σε MAPF**. CBS gives completeness αλλά exponential. Priority-based gives scalability αλλά incompleteness. Δεν υπάρχει free lunch — πρέπει να επιλέξεις βάσει deployment scenario.

Επίσης: **gossip protocols από distributed systems literature είναι ιδανικά για mobile robotics**. Slow dynamics (seconds-to-minute timescales), unreliable links, decentralized requirements — όλα ευνοούν epidemic algorithms over consensus protocols.

Η *novel framing* του risk ως shareable resource είναι το πιο interesting research contribution από αυτό το chapter — δείχνει πώς ιδέες από οικονομικά (resource allocation, auction mechanisms) μεταφέρονται σε ρομποτική.

Σύνδεση με Επόμενο

Στο Chapter 10 θα δούμε το Human-Aware Navigation. Σύνδεση: όταν fleets ρομπότ κινούνται σε spaces με ανθρώπους, multi-robot coordination σταντάρ + social proxemics + ethical zones = πλούσιο research area.

> **ΣΥΝΔΕΣΗ ΜΕ ΕΠΟΜΕΝΑ**
>
> • **C10 (Human-Aware)**: ρομπότ συντονίζονται respecting humans
> • **C14 (Causal SCM)**: deadlock root-cause analysis
> • **C08 (IDS)**: alarm propagation across fleet
> • **C03 (CVaR)**: individual risk budgets από distribution
> • **C05 (Safe-Mode)**: fleet-wide mode propagation

— Τέλος Κεφαλαίου 9 από 26 —

Συνέχισε με: ***Chapter 10 — Human-Aware Navigation: Ethical Zones, Proxemics, Trust***